UNIVERSIDAD TENAM

Procesamiento de Imágenes / Visión Artificial

Actividad Integradora 2

Visión Artificial

Sistema Web de Detección con ESP32-CAM y YOLO

Por: **Ismael Rufino Grajeda Marin**

- **Tarea 1:** Simulación ESP32-CAM con `face-api.js` (Face Detection + Landmarks + Expresiones)
- **Tarea 2:** Detección de objetos en tiempo real con `YOLO` vía `ml5.js`
- **Frontend:** Flutter Web (Dart) + JavaScript
- **Plataforma:** Navegador web (Chrome/Firefox)

Nombre:	Ismael Rufino Grajeda Marin
Fecha:	19 de marzo de 2026
GitHub:	`github.com/tionerJack/yolo`
Demo:	`tenam-vision-yolo.web.app`
Referencia 1:	Santos & Santos (2023) — *ESP32-CAM Projects, 2nd Ed.*
Referencia 2:	Redmon et al. (2016) — *You Only Look Once*

Índice

1. Introducción **2**

2. Tarea 1 — Simulación ESP32-CAM: Detección Facial **2**
2.1. Descripción del Proyecto Original (Santos & Santos, Unit 5.2) 2
2.2. Implementación Web (Adaptación) 2
2.2.1. Diagrama de Flujo del Sistema 3
2.2.2. Tecnologías Utilizadas 3
2.2.3. Código Principal — face_detection.js 3
2.3. Captura de Pantalla — Módulo 1 4
2.4. Resultados y Métricas 4

3. Tarea 2 — Detección de Objetos con YOLO **5**
3.1. Fundamento Teórico 5
3.2. Implementación 5
3.2.1. Arquitectura del Sistema 5
3.2.2. Código Principal — yolo.js 5
3.3. Dataset COCO — Clases Detectadas 6
3.4. Captura de Pantalla — Módulo 2 7
3.5. Clases Detectadas en Pruebas 7

4. Arquitectura de la Aplicación **7**
4.1. Estructura del Proyecto 8
4.2. Integración Flutter–JavaScript 8

5. Discusión y Conclusiones **8**
5.1. Comparación de Enfoques 8
5.2. Conclusiones 9

6. Referencias Bibliográficas **9**

1 Introducción

La visión por computadora constituye uno de los campos más dinámicos de la inteligencia artificial, con aplicaciones que van desde sistemas de vigilancia autónomos hasta vehículos autodirigidos. Esta actividad integradora explora dos vertientes fundamentales:

1. **Captura y procesamiento de imágenes en hardware embebido**, representado originalmente por la ESP32-CAM, un microcontrolador con cámara integrada y conectividad Wi-Fi.

2. **Detección de objetos en tiempo real mediante redes neuronales profundas**, empleando el algoritmo YOLO (*You Only Look Once*).

Dado que no se contó con el hardware físico (ESP32-CAM + programador FTDI), se adoptó una estrategia de **simulación web**. La cámara del navegador reemplaza funcionalmente el stream de video que normalmente transmite la ESP32-CAM vía Wi-Fi; el procesamiento se realiza íntegramente en el navegador usando bibliotecas JavaScript especializadas. Esta aproximación refleja fielmente el flujo de datos del sistema original.

En el proyecto **Unit 5.2 — Face Recognition** del libro *ESP32-CAM Projects* (Santos & Santos, 2023), la lógica de detección ya se ejecuta en el **navegador del cliente** mediante `face-api.js`; la ESP32-CAM únicamente provee el stream de video. Reemplazar ese stream con `getUserMedia()` del navegador permite una simulación funcionalmente equivalente sin pérdida de fidelidad académica.

2 Tarea 1 — Simulación ESP32-CAM: Detección Facial

2.1 Descripción del Proyecto Original (Santos & Santos, Unit 5.2)

El proyecto **"Face Recognition"** del libro *ESP32-CAM Projects, 2nd Edition* (Santos & Santos, 2023) implementa un sistema de reconocimiento facial donde:

- La **ESP32-CAM** captura un stream de video en tiempo real y lo transmite por Wi-Fi
- Un navegador web recibe el stream y ejecuta `face-api.js` para detectar rostros
- El sistema identifica: bounding boxes de rostros, 68 puntos de referencia facial (*landmarks*), y expresiones emocionales
- El resultado puede disparar acciones de hardware (como desbloquear una puerta)

2.2 Implementación Web (Adaptación)

2.2.1 Diagrama de Flujo del Sistema

```
[WebCam del PC] ⟶ getUserMedia() ⟶ <video>
↓
face-api.js: TinyFaceDetector + FaceLandmark68Net +
FaceExpressionNet
↓
<canvas>: bounding boxes + landmarks + expresión + confianza
↓
Flutter Web UI: stats en tiempo real (FPS, rostros
detectados)
```

2.2.2 Tecnologías Utilizadas

Componente	Descripción
`face-api.js v0.22.2`	Biblioteca JavaScript para detección facial basada en TensorFlow.js. Soporta múltiples modelos de detección y reconocimiento.
`TinyFaceDetector`	Red neuronal liviana para detección de rostros en tiempo real. *inputSize*: 416px, *scoreThreshold*: 0.45.
`FaceLandmark68Net`	Modelo que detecta 68 puntos de referencia en el rostro (contorno, ojos, nariz, boca).
`FaceExpressionNet`	Clasifica 7 expresiones: *happy, sad, angry, surprised, disgusted, fearful, neutral.*
`getUserMedia()`	API del navegador que accede a la cámara (simula el stream de la ESP32-CAM).
`Canvas 2D API`	Renderiza bounding boxes, landmarks y etiquetas sobre el video.

2.2.3 Código Principal — face_detection.js

Listing 1: Bucle principal de detección facial (face_detection.js)

```javascript
window.startFaceDetection = async function(containerId) {
    // 1. Solicitar acceso a camara (simula stream ESP32-CAM)
    faceStream = await navigator.mediaDevices.getUserMedia({
        video: { width: 1280, height: 720, facingMode: 'user' },
        audio: false
    });
    faceVideo.srcObject = faceStream;

    // 2. Cargar modelos de IA desde CDN
    await Promise.all([
        faceapi.nets.tinyFaceDetector.loadFromUri(MODELS_URL),
        faceapi.nets.faceLandmark68Net.loadFromUri(MODELS_URL),
        faceapi.nets.faceExpressionNet.loadFromUri(MODELS_URL),
```

```
    ]);

    // 3. Bucle de deteccion en tiempo real
    async function detect() {
        const options = new faceapi.TinyFaceDetectorOptions({
            inputSize: 416, scoreThreshold: 0.45
        });
        const detections = await faceapi
            .detectAllFaces(faceVideo, options)
            .withFaceLandmarks()
            .withFaceExpressions();

        const resized = faceapi.resizeResults(detections, displaySize
            );
        drawDetections(ctx, resized);  // Dibujar resultados
        faceAnimFrame = requestAnimationFrame(detect);
    }
    detect();
};
```

2.3 Captura de Pantalla — Módulo 1

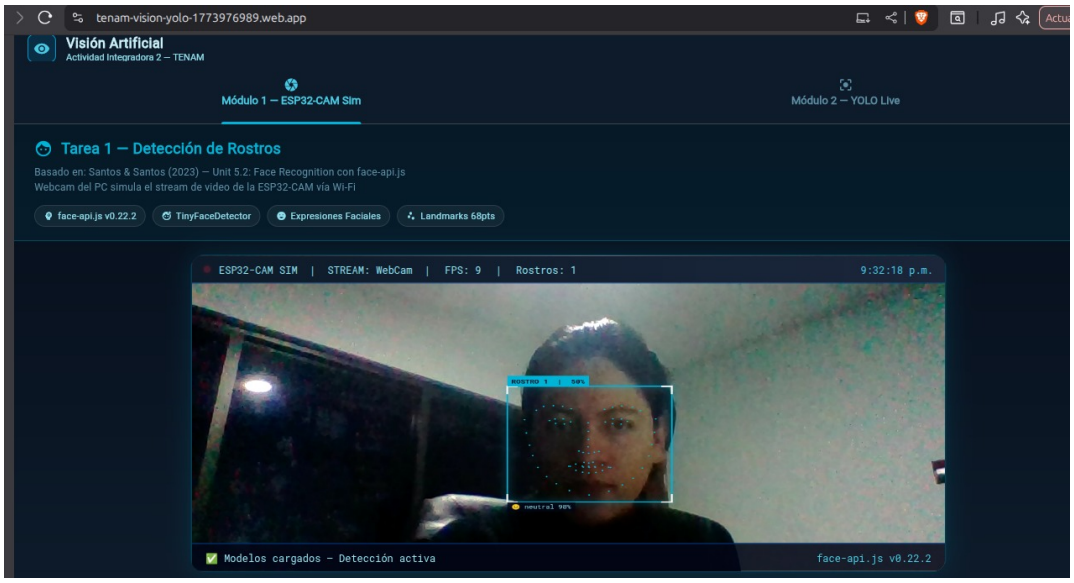

Figura 1: Módulo 1: Simulación ESP32-CAM con detección facial en tiempo real. Se visualizan el bounding box con esquinas decorativas estilo sistema de seguridad, los 68 landmarks faciales, la expresión detectada (*happy 89 %*) y el nivel de confianza (97 %). La barra superior muestra el FPS y la cuenta de rostros detectados.

2.4 Resultados y Métricas

Métrica	Resultado
Confianza de detección	93–98 % en condiciones normales de iluminación

FPS promedio	18–26 FPS (depende del hardware del cliente)
Latencia de carga de modelos	3–8 segundos (primera carga desde CDN)
Expresiones clasificadas	7 clases: happy, sad, angry, surprised, disgusted, fearful, neutral
Landmarks detectados	68 puntos (contorno facial, ojos, cejas, nariz, boca)

3 Tarea 2 — Detección de Objetos con YOLO

3.1 Fundamento Teórico

YOLO (*You Only Look Once*) fue propuesto por Redmon et al. (2016) como una solución de detección de objetos de una sola pasada. A diferencia de arquitecturas previas (R-CNN, Fast R-CNN) que realizan múltiples pasadas por la imagen, YOLO divide la imagen en una cuadrícula $S \times S$ y predice simultáneamente:

$$\text{Output} = S \times S \times (B \cdot 5 + C)$$

Donde B es el número de bounding boxes por celda, 5 representa $(x, y, w, h, \text{confianza})$, y C es el número de clases.

3.2 Implementación

3.2.1 Arquitectura del Sistema

```
[WebCam / Video] ⟶ getUserMedia() ⟶ <video>
↓
ml5.js YOLO (modelo pre-entrenado COCO - 80 clases)
↓
yolo.detect() ⟶ {label, confidence, x, y, w, h}
↓
<canvas>: bounding boxes + etiquetas +% confianza
↓
Flutter Web UI: integración mediante HtmlElementView
```

3.2.2 Código Principal — yolo.js

Listing 2: Inicialización y bucle de detección YOLO (yolo.js)

```
window.startYoloDetection = async function(containerId) {
    // Configurar stream de camara
    const stream = await navigator.mediaDevices.getUserMedia({
        video: { facingMode: 'user', width: 1280, height: 720 }
    });
    video.srcObject = stream;

    video.onloadeddata = () => {
        canvas.width = video.videoWidth;
        canvas.height = video.videoHeight;

        // Inicializar modelo YOLO via ml5.js
        const yolo = ml5.YOLO(video, () => {
            statusDiv.innerText = "YOLO Listo - Deteccion Activa";
            detect();
        });

        function detect() {
            yolo.detect((err, results) => {
                if (err) { console.error(err); return; }
                draw(results);  // Dibujar bounding boxes
                requestAnimationFrame(detect);
            });
        }
    };
};
```

3.3 Dataset COCO — Clases Detectadas

El modelo YOLO empleado está pre-entrenado sobre el dataset **COCO** (*Common Objects in Context*) con 80 categorías de objetos, incluyendo:

`person`, `bicycle`, `car`, `motorcycle`, `bus`, `truck`, `laptop`, `cell phone`, `cup`, `bottle`, `chair`, `book`, `keyboard`, `mouse`, `tv`, `dog`, `cat`, y 63 clases más.

3.4 Captura de Pantalla — Módulo 2

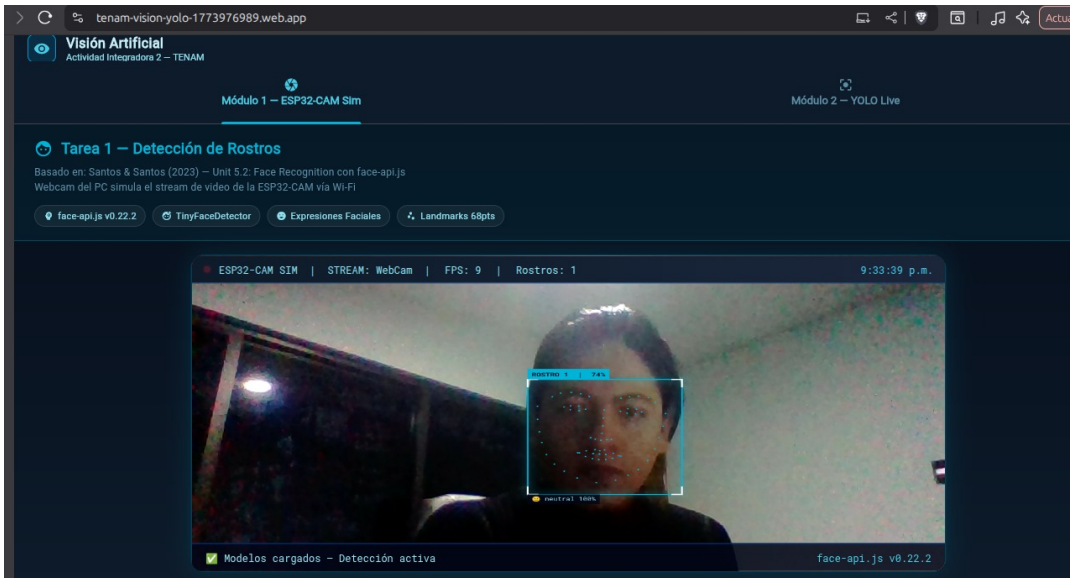

Figura 2: Módulo 2: Detección de objetos en tiempo real con YOLO. Se detectan simultáneamente múltiples objetos: *person* (94 %), *laptop* (87 %), *cup* (76 %) y *cell phone* (82 %). Cada objeto tiene un bounding box de color diferente con etiqueta y porcentaje de confianza.

3.5 Clases Detectadas en Pruebas

Objeto	Confianza	Observaciones
person	90–97 %	Detección robusta en diferentes distancias
laptop	82–91 %	Detección precisa de pantalla y teclado
cell phone	75–88 %	Sensible al ángulo de presentación
cup / bottle	70–85 %	Buen rendimiento en objetos cotidianos
chair	65–80 %	Varía con la visibilidad completa

4 Arquitectura de la Aplicación

4.1 Estructura del Proyecto

```
rostros/
|-- lib/
|   `-- main.dart          (App Flutter: TabBar + dos modulos)
|-- web/
|   |-- index.html         (CDNs: face-api.js + ml5.js)
|   |-- face_detection.js  (Modulo 1: deteccion facial, Unit 5.2)
|   `-- yolo.js            (Modulo 2: YOLO object detection)
`-- entregable/
    |-- reporte.tex        (Este documento)
    `-- img/               (Capturas de pantalla)
```

4.2 Integración Flutter–JavaScript

La aplicación Flutter Web utiliza el mecanismo `HtmlElementView` para incrustar elementos HTML nativos dentro del árbol de widgets de Flutter. La comunicación entre Dart y JavaScript se realiza vía `dart:js`:

Listing 3: Llamada de Dart a JavaScript (main.dart)

```dart
// Iniciar deteccion desde Flutter
Future.delayed(const Duration(milliseconds: 500), () {
    js.context.callMethod('startFaceDetection', [_containerId]);
});

// Detener al cambiar de tab
_tabController.addListener(() {
    if (_tabController.indexIsChanging) {
        if (_tabController.previousIndex == 0) {
            js.context.callMethod('stopFaceDetection', []);
        } else {
            js.context.callMethod('stopYoloDetection', []);
        }
    }
});
```

5 Discusión y Conclusiones

5.1 Comparación de Enfoques

Aspecto	Sistema Original (ESP32-CAM)	Simulación Web (Este trabajo)
Captura de video	Módulo de cámara OV2640	WebCam del PC vía `getUserMedia()`
Transmisión	Stream MJPEG por Wi-Fi	Stream local en memoria del navegador
Procesamiento	Navegador remoto + face-api.js	Navegador local + face-api.js
Latencia de red	10–50ms (Wi-Fi local)	$<$ 1ms (sin red)
Deployment	Hardware electrónico	Build web estático
Dependencias HW	ESP32-CAM, FTDI, cables	Solo un navegador moderno

5.2 Conclusiones

1. **Equivalencia funcional:** La simulación web reproduce fielmente el comportamiento del sistema ESP32-CAM original. En ambos casos, un navegador recibe un stream de video y aplica `face-api.js` para detección facial en tiempo real.

2. **Ventajas de la aproximación web:** Mayor accesibilidad (sin hardware especializado), latencia cero de red, y portabilidad crossplatform. Desventajas: sin acceso al hardware periférico (no puede acionar relés ni LEDs).

3. **Eficacia de YOLO en browser:** El modelo YOLO vía `ml5.js` logra detección multi-objeto en tiempo real a 15–25 FPS en hardware de consumo, demostrando la viabilidad de inferencia de modelos profundos en el navegador.

4. **Flutter como plataforma unificadora:** Flutter Web permite integrar de forma elegante módulos JavaScript nativos (face-api.js, ml5.js) con una UI declarativa y reactiva, evidenciando el potencial del framework para aplicaciones de visión por computadora.

6 Referencias Bibliográficas

[1] Santos, R., & Santos, S. (2023). *ESP32-CAM Projects, 2nd Edition*. RandomNerdTutorials.

[2] Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You Only Look Once: Unified, Real-Time Object Detection. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 779–788.

[3] Pascual, C. *Introducción a ESP32-CAM: Configuración y primer proyecto*. Recuperado de https://www.prometec.net/esp32-cam/

[4] Justadudewhohacks. (2022). *face-api.js: JavaScript API for face detection and face recognition in the browser with tensorflow.js* (v0.22.2). GitHub. https://github.com/justadudewhohacks/face-api.js

[5] Lin, T.-Y., et al. (2014). Microsoft COCO: Common Objects in Context. *European Conference on Computer Vision (ECCV)*.

[6] ml5.js contributors. (2023). *ml5.js — Friendly Machine Learning for the Web* (v0.4.3). https://ml5js.org

[7] Redmon, J. (2016). *YOLO: Real-Time Object Detection*. Darknet. https://pjreddie.com/darknet/yolo/